HW1 constant propagation & executor

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

constant propagation

该部分实现了对AST的遍历和转换，对于原始程序，先进行minus int rewrite，这一步已经写好了，直接使用。第二步就是进行constant propagation，遍历AST，将所有两边为常数的二元运算提前计算为常量，即常量折叠，最终得到一个新的clone的AST，对原来的AST不做任何改变。具体来说：

1. constantPropagate做了负整数重写，然后创建ConstantPropagation visitor对象 v，调用afterMinus->accept(v) 开始对AST进行遍历，返回变换后的program节点。
2. visitlist处理任何AST类型的节点列表，遍历每一个子节点x，调用x->accept(v)，最后将visitor返回的新节点保存到新列表。
3. visit(Program *node)遍历program的主函数节点main，递归调用accept(visitor)，cloneA节点并替换子节点构建一个新的program。
4. visit(MainMethod *node)遍历主函数语句列表，调用 visitList 递归处理每条语句然后生成新的MainMethod类型的节点。
5. visit(Assign *node)和visit(Return *node)比较类似，都是遍历对应类型语句的表达式子节点，调用accept(visitor)构造并保存新的对应类型子节点。
6. visit(BinaryOp *node)遍历它的左右操作数，如果都是常数，计算然后生成新的IntExp节点，进行了一次常量折叠。如果有变量，那么直接clone原来的节点。
7. visit(UnaryOp *node)，visit(IdExp *node)，visit(IntExp *node)，visit(OpExp *node)，都没有做常量折叠之类的处理，unary操作需要先遍历子节点，其他的都是直接clone节点。

execute the program

该部分是在执行AST程序，遍历整个AST然后计算程序结果，相当于实现了一个简单的解释器，使用了visitor模式。具体来说：

1. 先实现了evalBinaryOp()和evalUnaryOp()这两个用于进行一元计算和二元计算的工具函数。
2. execute()就是程序的入口，在执行AST时，同样的先root->accept(visitor)，然后根据节点类型调用对应的visit，返回结果。visit(Program)执行整个程序，进入main。visit(MainMethod)逐条执行main中的语句列表，到return就停止。visit(Assign)执行对变量的赋值，先检查左边是变量，然后计算右边的值，最后写入变量清空retVal。visit(Return)将表达式的值计算出来存入retVal中，检测它不为0就停止执行。visit(IntExp)读取整数常量，visit(IdExp)读取变量值，如果某个变量没有定

义，将它认为是0并打印极高2，汇报该变量所在的行和列。visit(BinaryOp)先计算左边，再计算右边，然后调用计算工具函数得到结果。visit(UnaryOp)计算右边，然后调用工具函数得到结果。visit(OpExp)直接retVal=0即可。

开发过程 & 测试结果

git graph截图：

Graph	Description	Date	Author	Commit
	master visitor pattern	5 Mar 2026 16:46	23300240034-chen...	42921d9e
	constant propagation and executor	5 Mar 2026 14:25	23300240034-chen...	4d0278a6
	origin/HEAD origin/master Slight modification	26 Feb 2026 15:36	王晓阳	55136e8a
	HW1 published	23 Feb 2026 00:20	王晓阳	30b4d729
	OK	22 Feb 2026 21:32	王晓阳	71e07708
	Remove README.md	22 Feb 2026 21:31	王晓阳	27abdb32
	Initialization	22 Feb 2026 21:30	王晓阳	392043f5
	Initial commit	22 Feb 2026 21:28	王晓阳	a8569e5c

部分测试结果截图：

test1：

```
root@cyx:~/fducompilerh2026/HW1/build# ./tools/main/main --debug ../test/test1
------Parsing fmj source file: ../test/test1.fmj------------
Convert AST  to XML...
Saving AST (XML) to: ../test/test1.2.ast
Loading AST (XML) from: ../test/test1.2.ast
Saving AST (XML) to: ../test/test1.2-debug.ast
Loading AST (XML) from: ../test/test1.2-debug.ast
Clone it ...
Convert cloned AST to XML...
Saving cloned AST (XML) to: ../test/test1.2-debug3.ast
Rewriting AST...
Convert rewrote AST to XML...
Saving AST (XML) to: ../test/test1.2-debug4.ast
Executing program...
Execution result: -7
-----Done---
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="x"/>
                <IntExp val="-7"/>
            </Assign>
            <Return>
                <IdExp id="x"/>
            </Return>
        </StmList>
    </MainMethod>
</Program>
```

test2：

```
root@cyx:~/fducompilerh2026/HW1/build# ./tools/main/main --debug ../test/test2
------Parsing fmj source file: ../test/test2.fmj------------
Convert AST  to XML...
Saving AST (XML) to: ../test/test2.2.ast
Loading AST (XML) from: ../test/test2.2.ast
Saving AST (XML) to: ../test/test2.2-debug.ast
Loading AST (XML) from: ../test/test2.2-debug.ast
Clone it ...
Convert cloned AST to XML...
Saving cloned AST (XML) to: ../test/test2.2-debug3.ast
Rewriting AST...
Convert rewrote AST to XML...
Saving AST (XML) to: ../test/test2.2-debug4.ast
Executing program...
Execution result: 2
-----Done---
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="x"/>
                <IntExp val="1"/>
            </Assign>
            <Assign>
                <IdExp id="y"/>
                <IntExp val="2"/>
            </Assign>
            <Assign>
                <IdExp id="z"/>
                <BinaryOp>
                    <IdExp id="x"/>
                    <OpExp op="+"/>
                    <IdExp id="y"/>
                </BinaryOp>
            </Assign>
            <Return>
                <BinaryOp>
                    <IdExp id="x"/>
                    <OpExp op="*"/>
                    <IdExp id="y"/>
                </BinaryOp>
            </Return>
        </StmList>
    </MainMethod>
</Program>
```

test4：

```
root@cyx:~/fducompilerh2026/HW1/build# ./tools/main/main --debug ../test/test4
------Parsing fmj source file: ../test/test4.fmj------------
Convert AST  to XML...
Saving AST (XML) to: ../test/test4.2.ast
Loading AST (XML) from: ../test/test4.2.ast
Saving AST (XML) to: ../test/test4.2-debug.ast
Loading AST (XML) from: ../test/test4.2-debug.ast
Clone it ...
Convert cloned AST to XML...
Saving cloned AST (XML) to: ../test/test4.2-debug3.ast
Rewriting AST...
Convert rewrote AST to XML...
Saving AST (XML) to: ../test/test4.2-debug4.ast
Executing program...
Execution result: -2
-----Done---
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="x"/>
                <IntExp val="-2"/>
            </Assign>
            <Return>
                <IdExp id="x"/>
            </Return>
        </StmList>
    </MainMethod>
</Program>
```

另外对于未定义变量但是使用的情况，测试时将tools/main/main.cc中的with_location_info变为true，或者将命令行的/main/main改成/test/test也行，得到的测试和结果如下：

test：

```
public int main() {
   x=1;
   z=(x+y);
   return (x*y);
}
```

```
root@cyx:~/fducompilerh2026/HW1/build# ./tools/main/main --debug ../test/test5
------Parsing fmj source file: ../test/test5.fmj------------
Convert AST  to XML...
Saving AST (XML) to: ../test/test5.2.ast
Loading AST (XML) from: ../test/test5.2.ast
Saving AST (XML) to: ../test/test5.2-debug.ast
Loading AST (XML) from: ../test/test5.2-debug.ast
Clone it ...
Convert cloned AST to XML...
Saving cloned AST (XML) to: ../test/test5.2-debug3.ast
Rewriting AST...
Convert rewrote AST to XML...
Saving AST (XML) to: ../test/test5.2-debug4.ast
Executing program...
Warning: variable 'y' used before definition at line 3, column 9
Execution result: 0
-----Done---
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Program bline="1" bpos="1" eline="5" epos="1">
    <MainMethod bline="1" bpos="1" eline="5" epos="1">
        <StmList>
            <Assign bline="2" bpos="4" eline="2" epos="7">
                <IdExp bline="2" bpos="4" eline="2" epos="4" id="x"/>
                <IntExp bline="2" bpos="6" eline="2" epos="6" val="1"/>
            </Assign>
            <Assign bline="3" bpos="4" eline="3" epos="11">
                <IdExp bline="3" bpos="4" eline="3" epos="4" id="z"/>
                <BinaryOp bline="3" bpos="6" eline="3" epos="10">
                    <IdExp bline="3" bpos="7" eline="3" epos="7" id="x"/>
                    <OpExp op="+" bline="3" bpos="8" eline="3" epos="8"/>
                    <IdExp bline="3" bpos="9" eline="3" epos="9" id="y"/>
                </BinaryOp>
            </Assign>
            <Return bline="4" bpos="4" eline="4" epos="16">
                <BinaryOp bline="4" bpos="11" eline="4" epos="15">
                    <IdExp bline="4" bpos="12" eline="4" epos="12" id="x"/>
                    <OpExp op="*" bline="4" bpos="13" eline="4" epos="13"/>
                    <IdExp bline="4" bpos="14" eline="4" epos="14" id="y"/>
                </BinaryOp>
            </Return>
        </StmList>
    </MainMethod>
</Program>

```

为了验证代码的确只爬了两遍树，在binaryOp之后的unaryOp不会再操作，进行下面的测试：

test：

```
public int main() {
    a = (-(1 + 2));
    return a;
}
```

```
root@cyx:~/fducompilerh2026/HW1/build# ./tools/main/main --debug ../test/test7
------Parsing fmj source file: ../test/test7.fmj------------
Convert AST  to XML...
Saving AST (XML) to: ../test/test7.2.ast
Loading AST (XML) from: ../test/test7.2.ast
Saving AST (XML) to: ../test/test7.2-debug.ast
Loading AST (XML) from: ../test/test7.2-debug.ast
Clone it ...
Convert cloned AST to XML...
Saving cloned AST (XML) to: ../test/test7.2-debug3.ast
Rewriting AST...
Convert rewrote AST to XML...
Saving AST (XML) to: ../test/test7.2-debug4.ast
Executing program...
Execution result: -3
-----Done---
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="a"/>
                <UnaryOp>
                    <OpExp op="-"/>
                    <IntExp val="3"/>
                </UnaryOp>
            </Assign>
            <Return>
                <IdExp id="a"/>
            </Return>
        </StmList>
    </MainMethod>
</Program>
```

参考资料

参考了lab1.md中的https://zhuanlan.zhihu.com/p/380161731，结合已经给我的minusIntConverter，学习了visitor pattern用来干什么和大概怎么干。

使用GPT帮我debug，主要是弄明白为什么test里面的main.cc不用把false改成true也能得到准确位置，但是main里面的mian.cc不行。